

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Analyzing and plotting the given data:

Given the files provided: agency.txt, stops.txt, routes.txt, trips.txt, stop_times.txt, calendar.txt, calendar_dates.txt and feed_info.txt

First, we need to check their contents and make them available in python with the helper: **gtfs_loader.py**

That loads all relevant GTFS tables into memory and exposes them as clean Python data structures.

```

1 class GTFSLoader:
2     """
3     Loads all relevant GTFS tables into memory and exposes them as clean Python
4     data structures.
5
6     Usage:
7         loader = GTFSLoader("path/to/gtfs_dir")
8         loader.load(date_str="20260303")
9     """
10
11 def __init__(self, gtfs_dir: str):
12     self.gtfs_dir = gtfs_dir
13
14     # Populated by load():
15     self.stops: dict = {}           # stop_id -> {name, lat, lon, location_type, parent_station}
16     self.routes: dict = {}         # route_id -> {name}
17     self.trips: dict = {}          # trip_id -> {route_id, service_id}
18     self.stop_times: dict = {}     # trip_id -> sorted list of stop-time dicts
19     self.active_services: set = set() # service_ids valid on the requested date

```

Each file has its own parser as each one has different attributes.

Example for the file stops.txt that represents the nodes of the graph:

```

1 def _load_stops(self):
2     """
3     stops.txt - every physical stop / platform in the network.
4
5     We distinguish:
6     location_type == 1 -> parent station (e.g. "Wrocław Główny")
7     location_type == 0 -> platform / specific boarding point
8     A platform may reference its parent via parent_station.
9     """
10    for row in _read_csv(os.path.join(self.gtfs_dir, "stops.txt")):
11        self.stops[row["stop_id"]] = {
12            "name": row["stop_name"].strip(),
13            "lat": float(row["stop_lat"]),
14            "lon": float(row["stop_lon"]),
15            "location_type": int(row.get("location_type")) or 0,
16            "parent_station": row.get("parent_station", "").strip(),
17        }

```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Once we have the data in dictionaries, we can start checking what we have by plotting the data into a map.

We need to create the program **visualize.py**

Thanks to the artificial intelligence for the coloring and styling of the html file and part of the comments in the code we can plot the stations (nodes) and trips (edges) into a map thanks to the coordinates provided in the data.

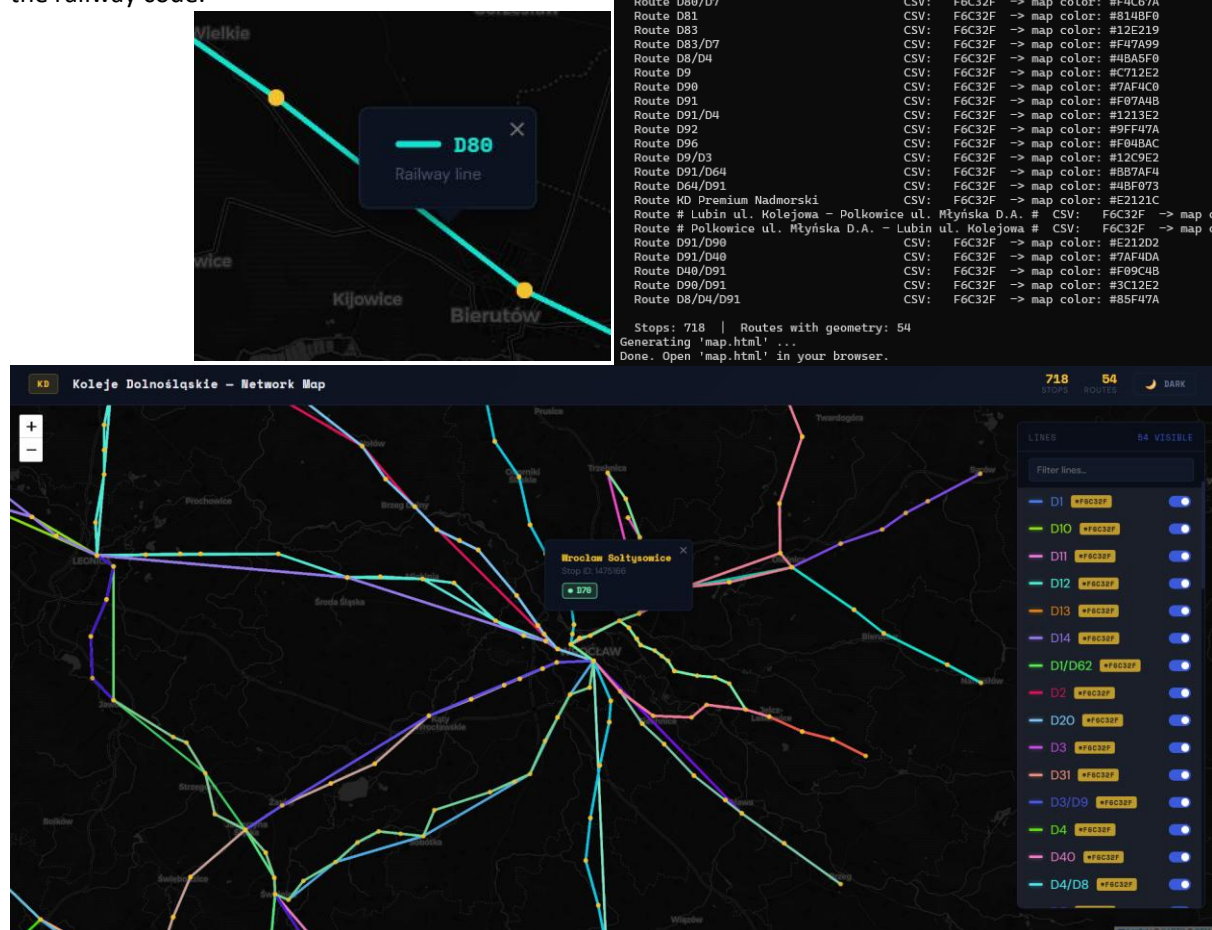
After running the command: **python visualize.py**

If all data is in its place, by default, the file **map.html** is created and added into the folder of the visualize.py file.

If you open it, all the station and lines will appear in a colorful way.

If you click in any station, you can check its name, id and also the trips/lines that pass through it.

As well as clicking on any colorful line will pop up the railway code.



Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

1. Searching algorithm as a cost function:

With the use of the provided GTFS dataset from Koleje Dolnośląskie, implement an algorithm searching for shortest paths between given stops A and B. As a cost function use (depending on the user's decision) travel time from A to B or the number of transfers needed.

Application should accept input in form of 4 variables:

- (a) the starting stop A
- (b) the ending stop B
- (c) optimization criterion: value t denotes travel time minimization, value p means minimization of the number of transfers
- (d) start time

The solution should print out to the standard output in successive lines detailed path information including starting stop, ending stop, used line's name, starting time, ending time, and on the standard error output the value of minimized criterion and time required for calculation of the shortest path.

a) Shortest path search from A to B using Dijkstra's algorithm, based on travel time criterion (10 points):

We need to implement the `dijkstra_time` function.

Why Dijkstra works here?

Even though edges have varying costs (wait + ride), the effective edge cost (`new_arrival - current_arrival`) is always ≥ 0 when we require `departure \geq arrival`.

This non-negativity is the only requirement for Dijkstra's correctness.

For making it work we need **for each station to calculate the best time to arrive to that stop given a `start_stop`** for that we store all reachable stops in a **`best_time`** dictionary and the previous best stop in the **`came_from`** dictionary.

Given that variables, we need to start with the algorithm by creating a priority queue for selecting the "best" node for continuing with the algorithm.

We create the final case where the current stop is equal to the end stop:

```
1 # best_time[stop_id] = minimum wall-clock arrival time at this stop
2 best_time: Dict[str, int] = {start_stop: start_time}
3
4 # came_from[(stop_id, time)] = (prev(stop_id, time), edge_with_from/to fields)
5 came_from: dict = {}
```

```
1 # Goal reached - reconstruct path
2 if stop_id == end_stop:
3     cost = curr_time - start_time
4     legs = _reconstruct(came_from, (stop_id, curr_time))
5     return cost, _format_legs(legs, graph.stops), nodes_explored
```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

And the important part of the Dijkstra algorithm is inside a loop with the condition that there is an element available in the heap we check if it is solution, if its time is worse than the best time to that station, if not, we perform the check of every edge/trip from that station if the train has not departed yet, and add the next stations into the heap:

So, if no path is found then it is impossible to get there and we return an empty answer.

```
1 return None, [], nodes_explored
```

```
1 for edge in graph.edges.get(stop_id, []):
2     if edge["type"] == "trip":
3         dep = edge["departure"]
4         arr = edge["arrival"]
5         nxt = edge["to"]
6
7         # We can only board if the train has not yet departed
8         if dep < curr_time:
9             continue
10
11         # New arrival time at the next stop
12         if arr < best_time.get(nxt, float("inf")):
13             best_time[nxt] = arr
14             enriched = {**edge, "from_stop": stop_id, "to_stop": nxt}
15             came_from[(nxt, arr)] = ((stop_id, curr_time), enriched)
16             heapq.heappush(pq, (arr, nxt, edge["trip_id"]))
17
18     elif edge["type"] == "transfer":
19         nxt = edge["to"]
20         new_time = curr_time + edge["transfer_time"]
21         if new_time < best_time.get(nxt, float("inf")):
22             best_time[nxt] = new_time
23             enriched = {**edge, "from_stop": stop_id, "to_stop": nxt,
24                         "departure": curr_time, "arrival": new_time}
25             came_from[(nxt, new_time)] = ((stop_id, curr_time), enriched)
26             heapq.heappush(pq, (new_time, nxt, None))
```

For checking that the algorithm works we can try it:

To run this part of task 1, we can run:

`python task1.py <start_stop> <end_stop> <criterion> <start_time> --algo dijkstra`

The command that I did as an example was: `python task1.py 1474651 1474981 t 08:00 --algo dijkstra`

```
Loading GTFS data from './gtfs' for date 20260303 ...
Stops: 1108 | Active trips: 767
Building transit graph ...
Edges: 12450

Routing: Wrocław Główny -> Cieśle
Depart after: 08:00:00 | Criterion: travel time | Algorithm: dijkstra
```

FROM	TO	LINE	DEP	ARR
Wrocław Główny	Wrocław Mikołajów	D9/D3	08:07:00	08:11:00
Wrocław Mikołajów	Wrocław Nadodrze	D83	08:44:00	08:48:00
Wrocław Nadodrze	Wrocław Sołtysowice	D83	08:49:00	08:53:00
Wrocław Sołtysowice	Wrocław Psie Pole	D83	08:54:00	08:56:00
Wrocław Psie Pole	Długoleka	D83	08:57:00	09:01:00
Długoleka	Borowa Oleśnicka	D80	10:17:00	10:22:00
Borowa Oleśnicka	Oleśnica	D80	10:22:00	10:29:00
Oleśnica	Cieśle	D81	11:49:00	12:01:00

```
Travel time: 4h 1m 0s (14460 seconds)
Calculation time: 0.007 s | Nodes explored: 942
```

We can manually check that the times are correct (in the trips.txt) and they are correct, let's check with another algorithm to see if part A is the fastest part.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

b) Shortest path search from A to B using A* algorithm, based on travel time criterion (25 points):

We need to implement the `astar_time` function, what heuristic was chosen?

$$h(n) = \frac{\text{haversine_distance}(n, \text{goal})}{\text{MAX_TRAIN_SPEED}}$$

```

1 R = 6_371_000.0
2 phi1, phi2 = math.radians(lat1), math.radians(lat2)
3 dphi = math.radians(lat2 - lat1)
4 dlam = math.radians(lon2 - lon1)
5 a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlam / 2) ** 2
6 return 2 * R * math.asin(math.sqrt(a))

```

And why this heuristic improves the performance of the solution improving dijkstra's time:

Dijkstra expands nodes in **all directions uniformly**. A* biases **expansion towards the goal** using the heuristic, so many stops in the wrong direction are never explored.

This dramatically reduces the number of nodes_explored.

Basically, instead of using from the heap the lowest time as in Dijkstra, we use the

$$f_{score} = elapsed\ time + heuristic\ remaining\ time$$

The code is exactly the same as in Dijkstra except for this parts:

The station extraction:

```

1 f, curr_time, stop_id, curr_trip = heapq.heappop(pq)

```

And the calculation in the push into the heap:

```

1 g = arr - start_time
2 h = graph.heuristic_time(nxt, end_stop)
3 heapq.heappush(pq, (g + h, arr, nxt, edge["trip_id"]))

```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Once that heuristic is applied, we can run the command:

`python task1.py <start_stop> <end_stop> <criterion> <start_time>`

So we run it with the previous parameters: **`python task1.py 1474651 1474981 t 08:00`**

```

Loading GTFS data from './gtfs' for date 20260303 ...
  Stops: 1108 | Active trips: 767
Building transit graph ...
  Edges: 12450

Routing: Wrocław Główny -> Cieśle
Depart after: 08:00:00 | Criterion: travel time | Algorithm: astar
FROM TO LINE DEP ARR
-----
Wrocław Główny Wrocław Mikołajów D9/D3 08:07:00 08:11:00
Wrocław Mikołajów Wrocław Nadodrze D83 08:44:00 08:48:00
Wrocław Nadodrze Wrocław Sołtysowice D83 08:49:00 08:53:00
Wrocław Sołtysowice Wrocław Psie Pole D83 08:54:00 08:56:00
Wrocław Psie Pole Długołęka D83 08:57:00 09:01:00
Długołęka Borowa Oleśnica D80 10:17:00 10:22:00
Borowa Oleśnica Oleśnica D80 10:22:00 10:29:00
Oleśnica Cieśle D81 11:49:00 12:01:00
Travel time: 4h 1m 0s (14460 seconds)
Calculation time: 0.009 s | Nodes explored: 858
  
```

In this case as the nodes are too near, Dijkstra's algorithm is better regarding the time it took, but the **A*** algorithm explored less nodes but with a larger time spent for each calculation of the heuristic value.

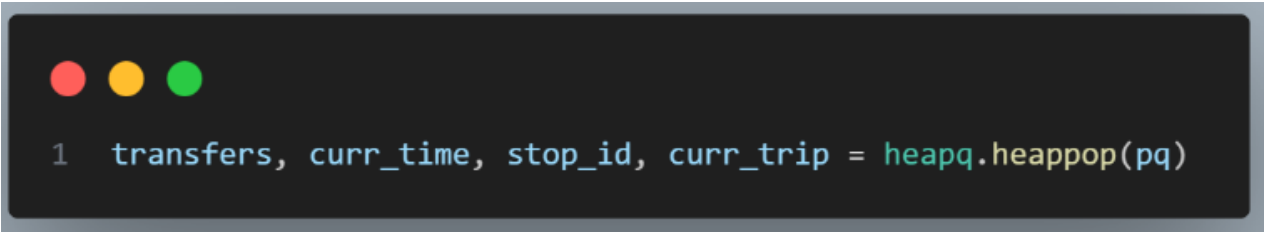
c) Shortest path search from A to B using A* algorithm, based on the number of transfers criterion (25 points):

We need to implement the `astar_transfers` function, what counts as a transfer?

Boarding the first train is free (no transfer). Every subsequent boarding of a **different** `trip_id` counts as +1 transfer. Walking between platforms of the same station does NOT add a transfer but does add waiting/walk time.

The selected heuristic as explained in the code is 0 for all nodes as it is simple and that a harder heuristic without a precomputation is hard to design.

Now the heap has the transfers as first:



```

1 transfers, curr_time, stop_id, curr_trip = heapq.heappop(pq)
  
```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

And the main logic:

```

1  new_transfers = transfers + (
2      1 if curr_trip is not None and tid != curr_trip else 0
3  )
4  new_time = arr
5  nxt_key = (nxt, tid)
6  bt2, bt2time = best.get(nxt_key, (float("inf"), float("inf")))
7
8  if new_transfers < bt2 or (new_transfers == bt2 and new_time < bt2time):
9      best[nxt_key] = (new_transfers, new_time)
10     enriched = {**edge, "from_stop": stop_id, "to_stop": nxt}
11     new_state = (nxt, new_time, tid)
12     prev_state = (stop_id, curr_time, curr_trip)
13     came_from[new_state] = (prev_state, enriched)
14     heapq.heappush(pq, (new_transfers, new_time, nxt, tid))

```

Once all is coded, we can perform the same search and check the differences:

python task1.py <start_stop> <end_stop> <criterion> <start_time>

So we run it with the previous parameters: **python task1.py 1474651 1474981 p 08:00**

```

Loading GTFS data from './gtfs' for date 20260303 ...
Stops: 1108 | Active trips: 767
Building transit graph ...
Edges: 12450

Routing: Wrocław Główny -> Cieśle
Depart after: 08:00:00 | Criterion: transfers | Algorithm: astar

```

FROM	TO	LINE	DEP	ARR
Wrocław Główny	Wrocław Mikołajów	D80	09:53:00	09:57:00
Wrocław Mikołajów	Wrocław Szczepin	D80	09:58:00	10:00:00
Wrocław Szczepin	Wrocław Nadodrze	D80	10:01:00	10:03:00
Wrocław Nadodrze	Wrocław Sołtysowice	D80	10:04:00	10:08:00
Wrocław Sołtysowice	Wrocław Psie Pole	D80	10:08:00	10:11:00
Wrocław Psie Pole	Mirków	D80	10:11:00	10:14:00
Mirków	Długołęka	D80	10:15:00	10:17:00
Długołęka	Borowa Oleśnicka	D80	10:17:00	10:22:00
Borowa Oleśnicka	Oleśnica	D80	10:22:00	10:29:00
Oleśnica	Cieśle	D81	11:49:00	12:01:00

```

Transfers: 1 transfer(s)
Calculation time: 0.175 s | Nodes explored: 11462

```

As you can see the number of trains was reduced to just 2, it took more time (due to the heuristic) but found a 1 transfer route that is better for the user to avoid changing trains.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

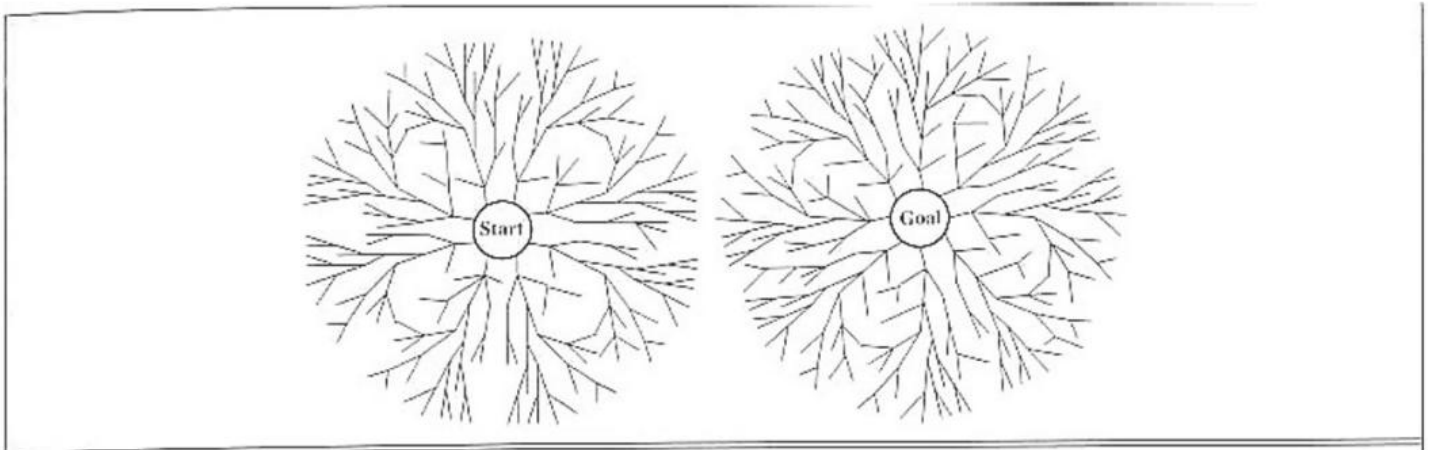
ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

d) Modifications of A* algorithm from (b) or (c), allowing the reduction of computational time or improving quality of found solution (10 points):

As told in theory, we can use a Bidirectional search with A*, usually is not necessary as A* is efficient enough but just for the experiment, let's do it.

I took inspiration from: https://github.com/ivanbgd/A-Star_Algorithm



The algorithm consists of **2 phases** as the graph is time dependent:

Phase 1: Start the algorithm in both start and end and find a meeting point.

Divided into the backward expansion:

```

1  else:
2      # -- Backward expansion (time-agnostic, pruning only) --
3      _, cost_b, stop_id = heapq.heappop(pq_b)
4      nodes_explored += 1
5
6      if stop_id in settled_b:
7          continue
8      settled_b.add(stop_id)
9
10     # Check if the forward search has already reached this stop.
11     if stop_id in best_f:
12         candidate = (best_f[stop_id] - start_time) + cost_b
13         if candidate < mu:
14             mu = candidate
15
16     # Expand static reverse edges
17     for rev_edge in backward_graph.get(stop_id, []):
18         nxt = rev_edge["to"]
19         new_cost = cost_b + rev_edge["travel_time"]
20         if new_cost < best_b.get(nxt, INF):
21             best_b[nxt] = new_cost
22             h = graph.heuristic_time(nxt, start_stop)
23             heapq.heappush(pq_b, (new_cost + h, new_cost, nxt))

```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

And the forward expansion:

```

1  if f_f_norm <= f_b_norm:
2      # -- Forward expansion --
3      _, curr_time, stop_id, curr_trip = heapq.heappop(pq_f)
4      nodes_explored += 1
5
6      if stop_id in settled_f:
7          continue
8      settled_f.add(stop_id)
9
10     # Check if the backward search has reached this stop.
11     # If yes, it is a candidate meeting point: cost = elapsed + best_b.
12     if stop_id in best_b:
13         candidate = (curr_time - start_time) + best_b[stop_id]
14         if candidate < mu:
15             mu = candidate
16
17     # Direct goal check: forward search reached the destination.
18     if stop_id == end_stop:
19         candidate = curr_time - start_time
20         if candidate < mu:
21             mu = candidate
22         break
23
24     # Expand time-dependent forward edges
25     for edge in graph.edges.get(stop_id, []):
26         if edge["type"] == "trip":
27             dep, arr, nxt = edge["departure"], edge["arrival"], edge["to"]
28             if dep < curr_time:
29                 continue
30             if arr < best_f.get(nxt, INF):
31                 best_f[nxt] = arr
32                 enriched = {**edge, "from_stop": stop_id, "to_stop": nxt}
33                 came_from_f[(nxt, arr)] = ((stop_id, curr_time), enriched)
34                 g = arr - start_time
35                 h = graph.heuristic_time(nxt, end_stop)
36                 heapq.heappush(pq_f, (start_time + g + h, arr, nxt, edge["trip_id"]))
37
38         elif edge["type"] == "transfer":
39             nxt = edge["to"]
40             new_time = curr_time + edge["transfer_time"]
41             if new_time < best_f.get(nxt, INF):
42                 best_f[nxt] = new_time
43                 enriched = {**edge, "from_stop": stop_id, "to_stop": nxt,
44                             "departure": curr_time, "arrival": new_time}
45                 came_from_f[(nxt, new_time)] = ((stop_id, curr_time), enriched)
46                 g = new_time - start_time
47                 h = graph.heuristic_time(nxt, end_stop)
48                 heapq.heappush(pq_f, (start_time + g + h, new_time, nxt, None))

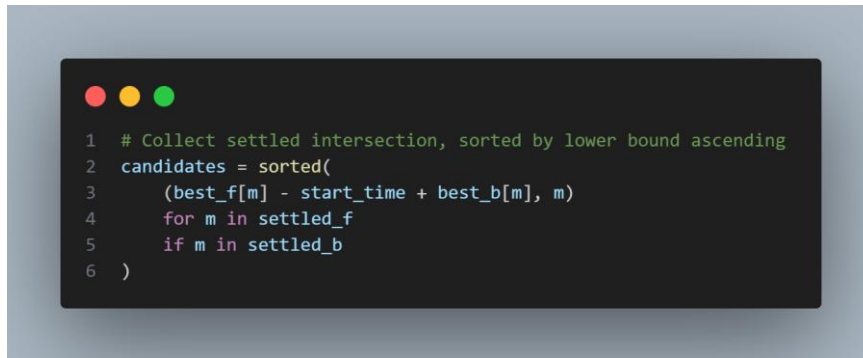
```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Phase 2: Once the algorithms meet, create another A* from the meeting point to the end to recalculate the times as the algorithm from the end to the meeting point couldn't calculate the exact time as it did not have any time reference of the end time.



The problem that I had was that it was working but as phase 2 was not implemented the solution was not efficient, so after adding phase 2 with the MAX_PHASE2_TRIES the result was now perfect and node exploration efficient.

Depart after: 08:00:00 Criterion: travel time Algorithm: bidir				
Building reverse graph for bidirectional search ...				
FROM	TO	LINE	DEP	ARR
Wrocław Główny	Wrocław Mikołajów	D9/D3	08:07:00	08:11:00
Wrocław Mikołajów	Wrocław Nadodrze	D7/D83	05:14:00	05:17:00
Wrocław Nadodrze	Wrocław Sołtysowice	D80	03:28:00	03:32:00
Wrocław Sołtysowice	Wrocław Psie Pole	D83	08:54:00	08:56:00
Wrocław Psie Pole	Długotąka	D80	03:49:00	03:53:00
Długotąka	Oleśnica	D80	03:54:00	04:04:00
Oleśnica	Cieśle	D81	04:09:00	04:19:00
Travel time: 0h 47m 0s (2820 seconds)				
Calculation time: 0.015 s Nodes explored: 90				

We can perform the same search and check the differences:

python task1.py <start_stop> <end_stop> <criteria> <start_time> --algo bidir

So we run it with the previous parameters: **python task1.py 1474651 1474981 p 08:00 --algo bidir**

Loading GTFS data from './gtfs' for date 20260303 ...				
Stops: 1108 Active trips: 767				
Building transit graph ...				
Edges: 12450				
Routing: Wrocław Główny -> Cieśle				
Depart after: 08:00:00 Criterion: travel time Algorithm: bidir				
Building reverse graph for bidirectional search ...				
FROM	TO	LINE	DEP	ARR
Wrocław Główny	Wrocław Mikołajów	D9/D3	08:07:00	08:11:00
Wrocław Mikołajów	Wrocław Nadodrze	D83	08:44:00	08:48:00
Wrocław Nadodrze	Wrocław Sołtysowice	D83	08:49:00	08:53:00
Wrocław Sołtysowice	Wrocław Psie Pole	D83	08:54:00	08:56:00
Wrocław Psie Pole	Długotąka	D83	08:57:00	09:01:00
Długotąka	Borowa Oleśnicka	D80	10:17:00	10:22:00
Borowa Oleśnicka	Oleśnica	D80	10:22:00	10:29:00
Oleśnica	Cieśle	D81	11:49:00	12:01:00
Travel time: 4h 1m 0s (14460 seconds)				
Calculation time: 0.023 s Nodes explored: 90				

The nodes explored are drastically reduced from the normal A*, so we can consider it a better solution with that approach.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

2. Several stops with the shortest path algorithm:

Using the provided GTFS dataset, implement the shortest path from a given stop A that visits all the stops given in a list $L = A_2, \dots, A_n$, and returns back to A. As a cost function use (depending on the user's decision) travel time from A to B or the number of transfers needed.

The application should accept input consisting of 4 variables:

- (a) starting stop A
- (b) list of stops to visit separated by a semicolon L
- (c) optimization criterion: value t denotes travel time minimization, value p means minimization of the number of transfers
- (d) starting time

The solution should print out to the standard output, in consecutive lines, the detailed path information including starting stop, ending stop, used line's name, starting time, ending time, and on the standard error output the value of the minimized criterion and time required for calculation of the shortest path.

a) An algorithm searching for the shortest path between the nodes based on Tabu Search without bounding the size of T (10 points):

First, we need to understand a little bit what the Tabu Search is, here is a useful link:

https://en.wikipedia.org/wiki/Tabu_search

We divide the code into 4 steps:

Stage 1: Calculating the shortest paths from every station to every station in the stops_list

```

1  cost_matrix: Dict[tuple, float] = {}
2  path_matrix: Dict[tuple, list] = {}
3
4  algo = astar_time if criterion == "t" else astar_transfers
5
6  for i, src in enumerate(stops_list):
7      for j, dst in enumerate(stops_list):
8          if i == j:
9              cost_matrix[(i, j)] = 0
10             path_matrix[(i, j)] = []
11             continue
12
13             cost, legs, _ = algo(graph, src, dst, start_time)
14
15             if cost is None:
16                 # No connection found - assign a very high penalty
17                 cost_matrix[(i, j)] = float("inf")
18                 path_matrix[(i, j)] = []
19             else:
20                 cost_matrix[(i, j)] = cost
21                 path_matrix[(i, j)] = legs
22
23  return cost_matrix, path_matrix

```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Stage 2: Applying the Tabu Search algorithm `tabu_search`

The algorithm steps are:

1. Generate initial solution (nearest-neighbour). -> `_nearest_neighbour_init(n_stops, cost_matrix)` and `_tour_cost(s, cost_matrix)`

```
1 unvisited = list(range(1, n_stops)) # indices 1..n (depot = 0)
2 current = 0
3 tour = []
4 while unvisited:
5     best_next = min(unvisited, key=lambda j: cost_matrix.get((current, j), float("inf")))
6     tour.append(best_next)
7     unvisited.remove(best_next)
8     current = best_next
9 return tour
```

```
1 n = len(perm)
2 total = cost_matrix[(0, perm[0])] # depot -> first stop
3 for k in range(n - 1):
4     total += cost_matrix[(perm[k], perm[k + 1])]
5 total += cost_matrix[(perm[-1], 0)] # last stop -> depot
6 return total
```

2. $s^* = s$, $T = \emptyset$.
3. Repeat for `step_limit` iterations:
 - a. Generate neighbourhood $N(s)$ (all 2-opt swaps, or sample).

```
1 new_perm = perm[:i] + perm[i:j + 1][::-1] + perm[j + 1:]
2 return new_perm
```

- b. Find best s' in $N(s) - T$ (or s' in T if aspiration criterion met).
 - c. Update T (FIFO, evict oldest if full).
 - d. $s = s'$.
 - e. If $D(s) < D(s^*)$: $s^* = s$.
4. Return s^* .

For an unbounded tabu list T , we use a set:

```
1 # Task 2a: unbounded - use a set for O(1) lookup
2 T_set: set = set()
3 T_deque: Optional[deque] = None
4
5 def _in_tabu(move: tuple) -> bool:
6     i, j = move
7     return move in T_set or (j, i) in T_set
8
9 def _add_tabu(move: tuple) -> None:
10     T_set.add(move)
```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

And finally, the main loop part of the code:

```

1  for (i, j) in pairs_to_try:
2      candidate = _two_opt_swap(s, i, j)
3      c_cost = _tour_cost(candidate, cost_matrix)
4      move = (i, j)
5
6      if _in_tabu(move):
7          pass
8      else:
9          # Non-tabu: valid candidate
10         if c_cost < best_candidate_cost:
11             best_candidate = candidate
12             best_candidate_cost = c_cost
13             best_move = move

```

If we found a better candidate, we make it the best and continue with another iteration.

b) Modification of the algorithm (a) with a variable T size, based on the length of L, in order to minimize the cost function (5 points):

We can design that if the `tabu_size` is written we apply this approach, otherwise the unbounded one:

```

1  # Task 2b: bounded FIFO deque
2  effective_size = max(7, tabu_size)
3  T_deque = deque(maxlen=effective_size)
4
5  def _in_tabu(move: tuple) -> bool:
6      i, j = move
7      return move in T_deque or (j, i) in T_deque
8
9  def _add_tabu(move: tuple) -> None:
10     T_deque.append(move)

```

This FIFO of bounded size will discard the oldest entry if it reaches its full capacity, and as we only change the functions content, the rest of the code remains the same.

The algorithms are: Caesar / R

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

- c) Modification of the algorithm (a) by adding Aspiration criterion, in order to minimize the cost function (5 points):

Just a simple piece of code can improve drastically the result:

```

1  if _in_tabu(move):
2      # Task 2c - aspiration: override tabu if this beats global best
3      if use_aspiration and c_cost < best_cost:
4          if c_cost < best_candidate_cost:
5              best_candidate = candidate
6              best_candidate_cost = c_cost
7              best_move = move

```

The aspiration prevents the tabu list from blocking genuine improvements. In it a **tabu move is accepted anyway if the resulting tour is better than the globally best-known tour s***.

- d) Modification of the algorithm (a) by adding a neighbor sampling strategy to minimize the cost function and reduce the computation time. (10 points):

For this part we need to create another input parameter to the tabu_search function called sampling and in the main loop of the tabu search we add:

```

1  # Task 2d - sampling: evaluate a random subset instead of all O(n^2)
2  if sampling and len(all_pairs) > sample_size:
3      pairs_to_try = random.sample(all_pairs, sample_size)
4  else: # Normal case: evaluate all pairs
5      pairs_to_try = all_pairs

```

This improves performance time by taking random samples of neighbours instead of all to avoid spending too much time on each one.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

After fixing a bug where the starting time was fixed once you reached a given destination and later about the pass of the day into the next stopped the algorithm from working.

This was the error code:

```
1 for a, b in zip(ordered_indices[:-1], ordered_indices[1:]):  
2     full_legs.extend(path_matrix.get((a, b), []))
```

And now the times are calculated fine.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

We can try all the modes of the code with:

Part A: python task2.py "1474651" "1474843;1475162;1475133" t 08:00 --no-aspiration

```

Depot : Wrocław Główny
Waypoints: Legnica -> Jelenia Góra -> Zgorzelec
Criterion: travel time
Start time: 08:00:00
Tabu size: unbounded/Task 2a | Steps: 300 | Aspiration: False | Sampling: False
[TSP] Stage 1: precomputing 12 pairwise paths ...
[TSP] Stage 2: Tabu Search steps=300 | T|=unbounded (Task 2a) aspiration=False sampling=False ...
[TSP] Stage 3: re-routing 4 legs with real times ...

--- Best tour found ---
Wrocław Główny -> Jelenia Góra -> Zgorzelec -> Legnica -> Wrocław Główny
FROM TO LINE DEP ARR
-----
Wrocław Główny Wrocław Grabiszyn D6 08:10:00 08:15:00
Wrocław Grabiszyn Wrocław Zachodni D6 08:16:00 08:17:00
Wrocław Zachodni Mokronos Górny D6 08:18:00 08:20:00
Mokronos Górny Smolec D6 08:21:00 08:23:00
Smolec Sadowice Wrocławskie D6 08:24:00 08:27:00
Sadowice Wrocławskie Kąty Wrocławskie D6 08:27:00 08:31:00
Kąty Wrocławskie Mietków D6 08:32:00 08:38:00
Mietków Imbramowice D6 08:38:00 08:43:00
Imbramowice Żarów D6 08:43:00 08:48:00
Żarów Jaworzyna Śląska D6 08:48:00 08:52:00
Jaworzyna Śląska Świebodzice D6 08:53:00 09:00:00
Świebodzice Wałbrzych Szczawienko D6 09:01:00 09:10:00
Wałbrzych Szczawienko Wałbrzych Miasto D6 09:12:00 09:16:00
Wałbrzych Miasto Wałbrzych Centrum D6 09:17:00 09:20:00
Wałbrzych Centrum Wałbrzych Fabryczny D6 09:20:00 09:24:00
Wałbrzych Fabryczny Wałbrzych Główny D6 09:24:00 09:30:00
Wałbrzych Główny Boguszków-Gorce Wschód D6 09:36:00 09:42:00
Boguszków-Gorce Wschód Boguszków-Gorce D6 09:42:00 09:44:00
Boguszków-Gorce Boguszków-Gorce Zachód D6 09:45:00 09:47:00
Boguszków-Gorce Zachód Witków Śląski D6 09:48:00 09:52:00
Witków Śląski Sędziszów D6 09:52:00 09:59:00
Sędziszów Marciszów D6 10:00:00 10:05:00
Marciszów Ciechanowice D6 10:05:00 10:08:00
Ciechanowice Janowice Wielkie D6 10:08:00 10:13:00
Janowice Wielkie Trzcianko D6 10:14:00 10:16:00
Trzcianko Wojanów D6 10:17:00 10:20:00
Wojanów Jelenia Góra D6 10:20:00 10:26:00
Jelenia Góra Jelenia Góra Zabobrze D62 11:45:00 11:47:00
Jelenia Góra Zabobrze Rybnica D62 11:47:00 11:56:00
Rybnica Stara Kamienica D62 11:57:00 12:03:00
Stara Kamienica Kwiecień D62 12:03:00 12:08:00
Kwiecień Rębiszów D62 12:09:00 12:14:00
Rębiszów Młynsko D62 12:14:00 12:20:00
Młynsko Gryfów Śląski D62 12:20:00 12:28:00
Gryfów Śląski Ubocze D62 12:34:00 12:37:00
Ubocze Olszyna Lubańska D62 12:37:00 12:42:00
Olszyna Lubańska Lubań Śląski D62 12:42:00 12:53:00
Lubań Śląski Zaręba D62 12:53:00 12:59:00
Zaręba Batowice Lubańskie D62 13:00:00 13:04:00
Batowice Lubańskie Studniska D62 13:05:00 13:11:00
Studniska Jerzmanki D62 13:11:00 13:15:00
Jerzmanki Zgorzelec D62 13:15:00 13:22:00
Zgorzelec Zgorzelec Miasto D10 14:17:00 14:19:00
Zgorzelec Miasto Jędrzychowice D10 14:20:00 14:22:00
Jędrzychowice Pieńsk D10 14:22:00 14:27:00
Pieńsk Węgliniec D10 14:28:00 14:36:00
Węgliniec Zagajnik D10 14:37:00 14:41:00
Zagajnik Zembrzydowa D10 14:41:00 14:46:00
Zembrzydowa Bolesławiec D10 14:46:00 14:54:00
Bolesławiec Tomaszów Bolesławiecki D10 14:55:00 15:00:00
Tomaszów Bolesławiecki Okmiany D10 15:01:00 15:05:00
Okmiany Osetnica D10 15:06:00 15:09:00
Osetnica Chojnów D10 15:10:00 15:14:00
Chojnów Miłkowice D10 15:15:00 15:21:00
Miłkowice Legnica D10 15:21:00 15:29:00
Legnica Malczyce D11 16:01:00 16:12:00
Malczyce Środa Śląska D11 16:13:00 16:18:00
Środa Śląska Miękinia D11 16:18:00 16:23:00
Miękinia Wrocław Leśnica D11 16:24:00 16:30:00
Wrocław Leśnica Wrocław Nowy Dwór D11 16:30:00 16:34:00
Wrocław Nowy Dwór Wrocław Muchobór D11 16:34:00 16:37:00
Wrocław Muchobór Wrocław Główny D11 16:37:00 16:43:00

Total travel time: 8h 55m 0s (32100 s)
Calculation time: 0.038 s

```

You can check that the Tabu size is unbounded with also aspiration and sampling off.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Part B: python task2.py "1474651" "1474843;1475162;1475133" t 08:00 --no-aspiration --tabu-size 5

```

Depot : Wrocław Główny
Waypoints: Legnica -> Jelenia Góra -> Zgorzelec
Criterion: travel time
Start time: 08:00:00
Tabu size: 5/Task 2b | Steps: 300 | Aspiration: False | Sampling: False
[TSP] Stage 1: precomputing 12 pairwise paths ...
[TSP] Stage 2: Tabu Search steps=300 |T|=max(7,5)=7 (Task 2b) aspiration=False sampling=False ...
[TSP] Stage 3: re-routing 4 legs with real times ...

--- Best tour found ---
Wrocław Główny -> Jelenia Góra -> Zgorzelec -> Legnica -> Wrocław Główny
FROM TO LINE DEP ARR
Wrocław Główny Wrocław Grabiszyn D6 08:10:00 08:15:00
Wrocław Grabiszyn Wrocław Zachodni D6 08:16:00 08:17:00
Wrocław Zachodni Mokronos Górny D6 08:18:00 08:20:00
Mokronos Górny Smolec D6 08:21:00 08:23:00
Smolec Sadowice Wrocławskie D6 08:24:00 08:27:00
Sadowice Wrocławskie Kąty Wrocławskie D6 08:27:00 08:31:00
Kąty Wrocławskie Mietków D6 08:32:00 08:38:00
Mietków Imbramowice D6 08:38:00 08:43:00
Imbramowice Żarów D6 08:43:00 08:48:00
Żarów Jaworzyna Śląska D6 08:48:00 08:52:00
Jaworzyna Śląska Świebodzice D6 08:53:00 09:00:00
Świebodzice Wałbrzych Szczawienko D6 09:01:00 09:10:00
Wałbrzych Szczawienko Wałbrzych Miasto D6 09:12:00 09:16:00
Wałbrzych Miasto Wałbrzych Centrum D6 09:17:00 09:20:00
Wałbrzych Centrum Wałbrzych Fabryczny D6 09:20:00 09:24:00
Wałbrzych Fabryczny Wałbrzych Główny D6 09:24:00 09:30:00
Wałbrzych Główny Boguszków-Gorce Wschód D6 09:36:00 09:42:00
Boguszków-Gorce Wschód Boguszków-Gorce D6 09:42:00 09:44:00
Boguszków-Gorce Boguszków-Gorce Zachód D6 09:45:00 09:47:00
Boguszków-Gorce Zachód Witków Śląski D6 09:48:00 09:52:00
Witków Śląski Sędziszów D6 09:52:00 09:59:00
Sędziszów Marciszów D6 10:00:00 10:05:00
Marciszów Ciechanowice D6 10:05:00 10:08:00
Ciechanowice Janowice Wielkie D6 10:08:00 10:13:00
Janowice Wielkie Trzcianko D6 10:14:00 10:16:00
Trzcianko Wojanów D6 10:17:00 10:20:00
Wojanów Jelenia Góra D6 10:20:00 10:26:00
Jelenia Góra Jelenia Góra Zabobrze D62 11:45:00 11:47:00
Jelenia Góra Zabobrze Rybnica D62 11:47:00 11:56:00
Rybnica Stara Kamienica D62 11:57:00 12:03:00
Stara Kamienica Kwieciszowice D62 12:03:00 12:08:00
Kwieciszowice Rębiszów D62 12:09:00 12:14:00
Rębiszów Młynsko D62 12:14:00 12:20:00
Młynsko Gryfów Śląski D62 12:20:00 12:28:00
Gryfów Śląski Ubocze D62 12:34:00 12:37:00
Ubocze Olszyna Lubańska D62 12:37:00 12:42:00
Olszyna Lubańska Luban Śląski D62 12:42:00 12:53:00
Luban Śląski Zaręba D62 12:53:00 12:59:00
Zaręba Batowice Lubańskie D62 13:00:00 13:04:00
Batowice Lubańskie Studniska D62 13:05:00 13:11:00
Studniska Jerzmanki D62 13:11:00 13:15:00
Jerzmanki Zgorzelec D62 13:15:00 13:22:00
Zgorzelec Zgorzelec Miasto D10 14:17:00 14:19:00
Zgorzelec Miasto Jędrzychowice D10 14:20:00 14:22:00
Jędrzychowice Pieńsk D10 14:22:00 14:27:00
Pieńsk Węgliniec D10 14:28:00 14:36:00
Węgliniec Zagajnik D10 14:37:00 14:41:00
Zagajnik Zebrzydowa D10 14:41:00 14:46:00
Zebrzydowa Bolesławiec D10 14:46:00 14:54:00
Bolesławiec Tomaszów Bolesławiecki D10 14:55:00 15:00:00
Tomaszów Bolesławiecki Okmiany D10 15:01:00 15:05:00
Okmiany Osetnica D10 15:06:00 15:09:00
Osetnica Chojnów D10 15:10:00 15:14:00
Chojnów Miłkowice D10 15:15:00 15:21:00
Miłkowice Legnica D10 15:21:00 15:29:00
Legnica Malczyce D11 16:01:00 16:12:00
Malczyce Środa Śląska D11 16:13:00 16:18:00
Środa Śląska Miękinia D11 16:18:00 16:23:00
Miękinia Wrocław Leśnica D11 16:24:00 16:30:00
Wrocław Leśnica Wrocław Nowy Dwór D11 16:30:00 16:34:00
Wrocław Nowy Dwór Wrocław Muchobór D11 16:34:00 16:37:00
Wrocław Muchobór Wrocław Główny D11 16:37:00 16:43:00

Total travel time: 8h 55m 0s (32100 s)
Calculation time: 0.038 s

```

We can check that the best solution is the same and as the number of edges per node is low, the time is similar to the unbounded strategy (Part A).

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Part C: python task2.py "1474651" "1474843;1475162;1475133" t 08:00

```

Depot : Wrocław Główny
Waypoints: Legnica -> Jelenia Góra -> Zgorzelec
Criterion: travel time
Start time: 08:00:00
Tabu size: unbounded/Task 2a | Steps: 300 | Aspiration: True | Sampling: False
[TSP] Stage 1: precomputing 12 pairwise paths ...
[TSP] Stage 2: Tabu Search steps=300 | T|=unbounded (Task 2a) aspiration=True sampling=False ...
[TSP] Stage 3: re-routing 4 legs with real times ...

--- Best tour found ---
Wrocław Główny -> Jelenia Góra -> Zgorzelec -> Legnica -> Wrocław Główny
FROM TO LINE DEP ARR
-----
Wrocław Główny Wrocław Grabiszyn D6 08:10:00 08:15:00
Wrocław Grabiszyn Wrocław Zachodni D6 08:16:00 08:17:00
Wrocław Zachodni Mokronos Górny D6 08:18:00 08:20:00
Mokronos Górny Smolec D6 08:21:00 08:23:00
Smolec Sadowice Wrocławskie D6 08:24:00 08:27:00
Sadowice Wrocławskie Kąty Wrocławskie D6 08:27:00 08:31:00
Kąty Wrocławskie Mietków D6 08:32:00 08:38:00
Mietków Imbramowice D6 08:38:00 08:43:00
Imbramowice Żarów D6 08:43:00 08:48:00
Żarów Jaworzyna Śląska D6 08:48:00 08:52:00
Jaworzyna Śląska Świebodzice D6 08:53:00 09:00:00
Świebodzice Wałbrzych Szczawienko D6 09:01:00 09:10:00
Wałbrzych Szczawienko Wałbrzych Miasto D6 09:12:00 09:16:00
Wałbrzych Miasto Wałbrzych Centrum D6 09:17:00 09:20:00
Wałbrzych Centrum Wałbrzych Fabryczny D6 09:20:00 09:24:00
Wałbrzych Fabryczny Wałbrzych Główny D6 09:24:00 09:30:00
Wałbrzych Główny Boguszków-Gorce Wschód D6 09:36:00 09:42:00
Boguszków-Gorce Wschód Boguszków-Gorce D6 09:42:00 09:44:00
Boguszków-Gorce Boguszków-Gorce Zachód D6 09:45:00 09:47:00
Boguszków-Gorce Zachód Witków Śląski D6 09:48:00 09:52:00
Witków Śląski Sędziszów D6 09:52:00 09:59:00
Sędziszów Marciszów D6 10:00:00 10:05:00
Marciszów Ciechanowice D6 10:05:00 10:08:00
Ciechanowice Janowice Wielkie D6 10:08:00 10:13:00
Janowice Wielkie Trzcianko D6 10:14:00 10:16:00
Trzcianko Wojanów D6 10:17:00 10:20:00
Wojanów Jelenia Góra D6 10:20:00 10:26:00
Jelenia Góra Jelenia Góra Zabobrze D62 11:45:00 11:47:00
Jelenia Góra Zabobrze Rybnica D62 11:47:00 11:56:00
Rybnica Stara Kamienica D62 11:57:00 12:03:00
Stara Kamienica Kwieciszowice D62 12:03:00 12:08:00
Kwieciszowice Rębiszów D62 12:09:00 12:14:00
Rębiszów Młynsko D62 12:14:00 12:20:00
Młynsko Gryfów Śląski D62 12:20:00 12:28:00
Gryfów Śląski Ubocze D62 12:34:00 12:37:00
Ubocze Olszyna Lubańska D62 12:37:00 12:42:00
Olszyna Lubańska Lubań Śląski D62 12:42:00 12:53:00
Lubań Śląski Zaręba D62 12:53:00 12:59:00
Zaręba Batowice Lubańskie D62 13:00:00 13:04:00
Batowice Lubańskie Studniska D62 13:05:00 13:11:00
Studniska Jerzmanki D62 13:11:00 13:15:00
Jerzmanki Zgorzelec D62 13:15:00 13:22:00
Zgorzelec Zgorzelec Miasto D10 14:17:00 14:19:00
Zgorzelec Miasto Jędrzychowice D10 14:20:00 14:22:00
Jędrzychowice Pieńsk D10 14:22:00 14:27:00
Pieńsk Węgliniec D10 14:28:00 14:36:00
Węgliniec Zagajnik D10 14:37:00 14:41:00
Zagajnik Zebrzydowa D10 14:41:00 14:46:00
Zebrzydowa Bolesławiec D10 14:46:00 14:54:00
Bolesławiec Tomaszów Bolesławiecki D10 14:55:00 15:00:00
Tomaszów Bolesławiecki Okmiany D10 15:01:00 15:05:00
Okmiany Osetnica D10 15:06:00 15:09:00
Osetnica Chojnów D10 15:10:00 15:14:00
Chojnów Miłkowice D10 15:15:00 15:21:00
Miłkowice Legnica D10 15:21:00 15:29:00
Legnica Malczyce D11 16:01:00 16:12:00
Malczyce Środa Śląska D11 16:13:00 16:18:00
Środa Śląska Miękinia D11 16:18:00 16:23:00
Miękinia Wrocław Leśnica D11 16:24:00 16:30:00
Wrocław Leśnica Wrocław Nowy Dwór D11 16:30:00 16:34:00
Wrocław Nowy Dwór Wrocław Muchobór D11 16:34:00 16:37:00
Wrocław Muchobór Wrocław Główny D11 16:37:00 16:43:00

Total travel time: 8h 55m 0s (32100 s)
Calculation time: 0.039 s

```

The code time execution is similar to the bounded tabu list in previous part (B).

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Part D: python task2.py "1474651" "1474843;1475162;1475133;1475015;1475245" t 08:00 --sampling --sample-size 15 --steps 500

```

Depot : Wrocław Główny
Waypoints: Legnica -> Jelenia Góra -> Zgorzelec -> Lubin -> Góra Śląska
Criterion: travel time
Start time: 08:00:00
Tabu size: unbounded/Task 2a | Steps: 500 | Aspiration: True | Sampling: True
[TSP] Stage 1: precomputing 30 pairwise paths ...
[TSP] Stage 2: Tabu Search steps=500 |T|=unbounded (Task 2a) aspiration=True sampling=True ...
[TSP] Stage 3: re-routing 6 legs with real times ...
[TSP] INFO: leg 6 to 'Wrocław Główny' wraps to next day (search from 08:11:00)

--- Best tour found ---
Wrocław Główny -> Góra Śląska -> Zgorzelec -> Lubin -> Legnica -> Jelenia Góra -> Wrocław Główny
FROM TO LINE DEP ARR
Wrocław Główny Wrocław Mikołajów D9/D3 08:07:00 08:11:00
Wrocław Mikołajów Wrocław Popowice D9/D3 08:12:00 08:14:00
Wrocław Popowice Wrocław Różanka D9/D3 08:14:00 08:16:00
Wrocław Różanka Wrocław Osobowice D9/D3 08:16:00 08:19:00
Wrocław Osobowice Wrocław Świnariy D9/D3 08:19:00 08:22:00
Wrocław Świnariy Szewce D9/D3 08:23:00 08:25:00
Szewce Pęgów D9/D3 08:25:00 08:28:00
Pęgów Oborniki Śląskie D9/D3 08:28:00 08:33:00
Oborniki Śląskie Osola D9/D3 08:34:00 08:37:00
Osola Skokowa D9/D3 08:38:00 08:41:00
Skokowa Żmigród D9/D3 08:42:00 08:48:00
Żmigród Garbce D9/D3 08:49:00 08:52:00
Garbce Korzeńsko D9/D3 08:52:00 08:55:00
Korzeńsko Rawicz D9/D3 08:55:00 09:01:00
Rawicz Bojanowo D31 09:55:00 10:16:00
Bojanowo Zaborowice D31 10:16:00 10:22:00
Zaborowice Borszyn Wielki D31 10:22:00 10:27:00
Borszyn Wielki Borszyn Mały D31 10:27:00 10:30:00
Borszyn Mały Góra Śląska D31 10:30:00 10:39:00
Góra Śląska Borszyn Mały D31 11:38:00 11:44:00
Borszyn Mały Borszyn Wielki D31 11:44:00 11:47:00
Borszyn Wielki Zaborowice D31 11:47:00 11:52:00
Zaborowice Bojanowo D31 11:52:00 12:01:00
Bojanowo Rawicz D31 12:01:00 12:24:00
Rawicz Korzeńsko D3/D9 13:25:00 13:31:00
Korzeńsko Garbce D3/D9 13:31:00 13:34:00
Garbce Żmigród D3/D9 13:34:00 13:37:00
Żmigród Skokowa D3/D9 13:38:00 13:44:00
Skokowa Osola D3/D9 13:44:00 13:47:00
Osola Oborniki Śląskie D3/D9 13:48:00 13:51:00
Oborniki Śląskie Pęgów D3/D9 13:52:00 13:56:00
Pęgów Szewce D3/D9 13:57:00 13:59:00
Szewce Wrocław Świnariy D3/D9 14:00:00 14:02:00
Wrocław Świnariy Wrocław Osobowice D3/D9 14:02:00 14:06:00
Wrocław Osobowice Wrocław Różanka D3/D9 14:06:00 14:09:00
Wrocław Różanka Wrocław Popowice D3/D9 14:09:00 14:11:00
Wrocław Popowice Wrocław Mikołajów D3/D9 14:11:00 14:13:00
Wrocław Mikołajów Wrocław Główny D3/D9 14:14:00 14:19:00
Wrocław Główny Wrocław Muchobór D10 14:45:00 14:50:00
Wrocław Muchobór Wrocław Nowy Dwór D10 14:50:00 14:53:00
Wrocław Nowy Dwór Wrocław Żerniki D10 14:54:00 14:56:00
Wrocław Żerniki Wrocław Leśnica D10 14:56:00 14:59:00
Wrocław Leśnica Mrozów D10 15:00:00 15:05:00
Mrozów Miękinia D10 15:05:00 15:08:00
Miękinia Przedmoście Święte D10 15:09:00 15:12:00
Przedmoście Święte Środa Śląska D10 15:13:00 15:16:00
Środa Śląska Malczyce D10 15:17:00 15:22:00
Malczyce Szczedrzykowice D10 15:29:00 15:35:00
Szchedrzykowice Jaśkowice Legnickie D10 15:35:00 15:38:00
Jaśkowice Legnickie Kunice D10 15:38:00 15:41:00

```

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 1

Strzegom	Stanowice	D91/D40	22:25:00	22:29:00
Stanowice	Jaworzyna Śląska	D91/D40	22:29:00	22:35:00
Jaworzyna Śląska	Świebodzice	D6	22:40:00	22:47:00
Świebodzice	Wałbrzych Szczawienko	D6	22:47:00	22:57:00
Wałbrzych Szczawienko	Wałbrzych Miasto	D6	22:58:00	23:02:00
Wałbrzych Miasto	Wałbrzych Centrum	D6	23:04:00	23:07:00
Wałbrzych Centrum	Wałbrzych Fabryczny	D6	23:07:00	23:12:00
Wałbrzych Fabryczny	Wałbrzych Główny	D6	23:12:00	23:18:00
Wałbrzych Główny	Boguszków-Gorce Wschód	D6	23:20:00	23:26:00
Boguszków-Gorce Wschód	Boguszków-Gorce	D6	23:27:00	23:29:00
Boguszków-Gorce	Boguszków-Gorce Zachód	D6	23:29:00	23:32:00
Boguszków-Gorce Zachód	Witków Śląski	D6	23:33:00	23:37:00
Witków Śląski	Sędziszów	D6	23:38:00	23:43:00
Sędziszów	Marciszów	D6	23:44:00	23:49:00
Marciszów	Ciechanowice	D6	23:50:00	23:52:00
Ciechanowice	Janowice Wielkie	D6	23:53:00	23:58:00
Janowice Wielkie	Trzcińsko	D6	23:58:00	24:01:00
Trzcińsko	Wojanów	D6	24:02:00	24:05:00
Wojanów	Jelenia Góra	D6	24:05:00	24:11:00
Jelenia Góra	Wojanów	D6	03:48:00	03:53:00
Wojanów	Trzcińsko	D6	03:53:00	03:56:00
Trzcińsko	Janowice Wielkie	D6	03:57:00	03:59:00
Janowice Wielkie	Ciechanowice	D6	04:00:00	04:05:00
Ciechanowice	Marciszów	D6	04:05:00	04:08:00
Marciszów	Sędziszów	D6	04:08:00	04:14:00
Sędziszów	Witków Śląski	D6	04:15:00	04:20:00
Witków Śląski	Boguszków-Gorce Zachód	D6	04:21:00	04:25:00
Boguszków-Gorce Zachód	Boguszków-Gorce	D6	04:25:00	04:28:00
Boguszków-Gorce	Boguszków-Gorce Wschód	D6	04:28:00	04:30:00
Boguszków-Gorce Wschód	Wałbrzych Główny	D6	04:31:00	04:37:00
Wałbrzych Główny	Wałbrzych Fabryczny	D6	04:38:00	04:43:00
Wałbrzych Fabryczny	Wałbrzych Centrum	D6	04:46:00	04:49:00
Wałbrzych Centrum	Wałbrzych Miasto	D6	04:50:00	04:53:00
Wałbrzych Miasto	Wałbrzych Szczawienko	D6	04:54:00	04:58:00
Wałbrzych Szczawienko	Świebodzice	D6	04:59:00	05:08:00
Świebodzice	Jaworzyna Śląska	D6	05:08:00	05:15:00
Jaworzyna Śląska	Żarów	D6	05:16:00	05:20:00
Żarów	Imbramowice	D6	05:20:00	05:25:00
Imbramowice	Mietków	D6	05:25:00	05:30:00
Mietków	Kąty Wrocławskie	D6	05:30:00	05:36:00
Kąty Wrocławskie	Sadowice Wrocławskie	D6	05:37:00	05:40:00
Sadowice Wrocławskie	Smolec	D6	05:41:00	05:44:00
Smolec	Mokronos Górny	D6	05:45:00	05:47:00
Mokronos Górny	Wrocław Zachodni	D6	05:47:00	05:50:00
Wrocław Zachodni	Wrocław Grabiszyn	D6	05:50:00	05:52:00
Wrocław Grabiszyn	Wrocław Główny	D6	05:53:00	06:00:00
Total travel time: 22h 15m 0s (80100 s)				
Calculation time: 0.142 s				

After a fix in the way that the program manages the pass of the day (> 23:59:59), we can finally see that the algorithm can handle several stops in a reasonable amount of time.

Some of the comments in the code are made by AI (ChatGPT) to not spent lots of hours in them. :)